Multi-Group Pipe Routing under Permanent Geometric Occupancy: Problem, Benchmark, and Classical Baselines

Deng

Abstract

Additive manufacturing (AM) enables compact hydraulic components whose internal fluid channels can follow free-form 3D paths rather than conventionally drilled holes. A representative case is multi-group channel layout in rotary direct-drive servo valves: once a channel is placed it permanently occupies volume, so later channels must clear earlier geometry—unlike classical multi-agent pathfinding (MAPF), where agents free space after moving. We study this setting as *multi-group pipe routing under permanent geometric occupancy*. Our contributions are a problem formalization with geometric dual-witness conflicts, a constructive 3D benchmark (two corridor generators $\times$ two obstacle painters, controlled difficulty, feasibility witnesses), and baseline results for CBS, PBS, and priority planning adapted to this coupling. We evaluate by success rate under a fixed time budget. PlaneSlice hard instances clearly rank CBS $\gg$ PBS $\gg$ PP, while easier cells validate the pipeline. The goal is a reproducible problem definition, suite, and classical baselines—not a new optimal MAPF algorithm.

1 Introduction and Motivation

Conventional hydraulic valves often realize internal flow paths by intersecting drilled holes. Additive manufacturing removes much of that machining constraint: channels can be shorter, smoother, and spatially interleaved inside a dense envelope, which is attractive for reducing pressure loss and package size [**?**].

A motivating device is the *high-performance rotary direct-drive servo valve*. Structurally it combines:

- a **center axis core** driven by a motor (rotary spool / sleeve interface);
- several **groups of external terminals** (ports) on the valve body;
- **non-conflicting fluid pipes** that connect terminals of each group toward the center axis (and related internal openings).

At different rotation angles, different groups are brought into hydraulic connection through the rotary core. Therefore pipes belonging to *different* groups must remain geometrically separated for all design poses of interest, while pipes *within* a group may share trunks or branch (tree-like manifolds), as illustrated in Fig. 1.

AM makes such layouts manufacturable, but *designing* the channels is still a combinatorial 3D routing problem: many terminals, cluttered obstacles, and hard clearance between groups. Manual CAD routing does not scale; we seek automatic planners that return conflict-free multi-group layouts inside a bounded design domain.

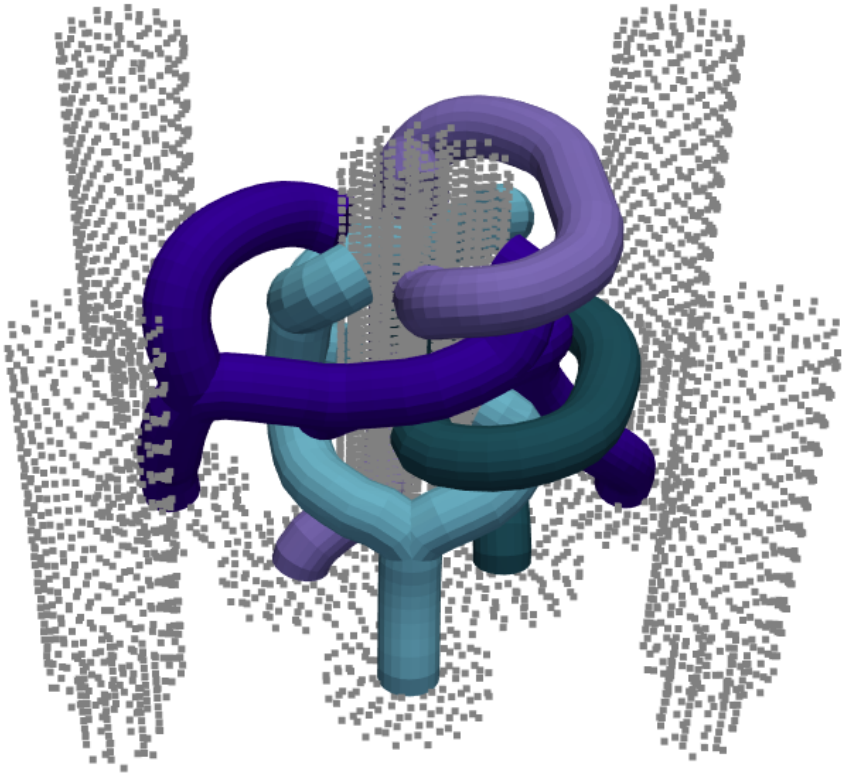


Figure 1: Illustrative multi-group fluid channels around a center axis and structural pillars (visualisation of an AM-oriented valve-style layout). Coloured tubes are pipes of different groups; grey volumes mark occupied / forbidden structure. Channels are short and smoothly curved relative to drilled manifolds, but must not collide.

Contributions.

1. **Problem.** We formalize multi-group pipe routing under permanent geometric occupancy, with dual-witness clearance conflicts and topology-driven multi-terminal groups (Sec. 2).
2. **Benchmark.** We construct a synthetic 3D suite with known-feasible witnesses, two corridor generators, two obstacle painters, and easy/middle/hard schedules (Sec. 4).
3. **Baselines.** We adapt CBS, PBS, and priority planning to this stack and report fixed-budget success rates on 36 batches of N=100 instances each (Secs. 3–6).

We do not claim a new optimal high-level algorithm or industrial CAD parity.

2 From Valve Design to a Computational Problem

We abstract the valve setting as *multi-group pipe routing under permanent geometric occupancy*. The design domain is a bounded 3D region with static obstacles (housing, fasteners, sealed volumes, axis envelope, ...). Terminals are partitioned into groups that may share a manifold toward the axis; each routed pipe permanently occupies a tubular neighbourhood of radius r (there is no "pass and release" as in robot MAPF); cross-group geometric interference is forbidden, while within-group

sharing / branching is allowed via a topology plan. This combination is not captured by classical MAPF suites, so we formalize the problem below and construct a synthetic benchmark (Sec. 4).

Geometry and occupancy. The domain is discretised as a regular 3D grid of resolution Δ (we use Δ=1 without loss of generality by scaling). A pipe of radius r occupies a *capsule* neighbourhood along its centreline (sphere swept along each segment). Static obstacles are modelled as spheres, capsules, and/or axis-aligned boxes. A query point (or cell centre) p with pipe radius r is **free** iff its clearance to all static obstacles and to already committed pipes of *other* groups is at least r. Equivalently, the open ball $B(p, r)$ must not intersect forbidden geometry.

Groups, terminals, and topology. An instance specifies m groups $G_1, \ldots, G_m$. Group G_i has terminals $T_i = \{t_{i,1}, \ldots, t_{i,n_i}\}$ on the domain boundary or on prescribed interfaces such as the axis sleeve (ports in the valve metaphor; center-axis openings may be additional terminals or topology goals). Before geometric search, a topology builder maps T_i to tasks (edges) E_i—typically a spanning tree—and an optional intra-group growth order. Solving all tasks of G_i yields a connected tree-shaped network. Cross-group tasks interact only through occupancy.

Conflicts. Unlike classical MAPF vertex/edge conflicts at a shared time, two pipes of different groups conflict when their tubular neighbourhoods violate clearance. A detected conflict stores witness points (p_a, r_a) and (p_b, r_b) (generally $p_a \neq p_b$) and a deficit measuring penetration. Same-group contacts are not conflicts under the default sharing policy.

Feasible solution and objectives. A feasible solution is a set of paths for all tasks of all groups such that every path is free w.r.t. static obstacles and all other-group committed geometry. The primary experimental objective in this paper is *feasibility under a wall-clock budget* (success rate). Secondary signals include total path cost relative to a constructive witness. Manufacturing-oriented criteria (bend count, supportability, pressure-loss proxies) matter for valves but are left to future work; we focus on the geometric multi-group core first.

Table 1: Classical MAPF vs. multi-group pipe routing under permanent occupancy.

Aspect	Classical MAPF	This problem
Occupancy	Temporary (agents move)	Permanent (pipes stay)
Conflict	Same cell / edge @ time t	Geometric clearance (dual witnesses)
Agent	Single s–t path	Multi-terminal **group** (tree)
Time	Explicit time dimension	Design-time layout (no schedule)

Relation to MAPF. As shown in Table 1, the settings differ in occupancy, conflict semantics, and agent structure. High-level MAPF controllers (CBS, PBS, priority planning) remain useful as search over leaves (constraint sets or orders), provided low-level search and conflict detection use permanent geometric occupancy. We do not claim a drop-in replacement for spatiotemporal MAPF benchmarks.

3 Baseline Planner Stack

We adapt CBS, PBS, and priority planning (PP) to permanent geometric occupancy and multi-terminal groups. The shared stack comprises four layers:

Topology. Terminals $\{T_i\}$ map to tasks (edges) and an optional intra-group growth order. Reported runs use a Euclidean *pure MST* per group: same-group routed segments become connectable goals, forming a tree-shaped manifold. Cross-group interaction is only through occupancy.

Occupancy. Static geometry uses a clearance grid; committed other-group pipes and CBS negative constraints use an *obstacle chain* of spheres and capsules. A cell centre c is free for radius r iff clearance to every forbidden volume is at least r.

Low-level search. Each task is solved by grid A* (26-connected, Manhattan) on the composite grid, yielding a free cell polyline. Under PP the path is committed as hard occupancy for later groups; under CBS/PBS it is held in the current leaf solution.

Conflict detection. After all groups are routed in a leaf, we detect cross-group geometric hits as dual witnesses (p_a, r_a), (p_b, r_b) with a deficit (same-group contacts ignored). CBS/PBS rank conflicts with a *near-terminal* heuristic when choosing which hit to resolve.

3.1 High-level controllers

CBS. Constraint-tree leaves store per-group paths plus negative witness spheres on the chain. A selected conflict adds opposing constraints; only the affected group is replanned.

PBS. Leaves store a priority DAG over groups. A conflict adds a priority edge; the lower-priority group replans with higher-priority paths as hard obstacles (reported: depth-first expansion).

PP. A total order over groups is searched: prior groups are hard obstacles for later ones; failure triggers sequence neighbourhood repair. There is no dual-witness constraint branching—only order search.

4 Benchmark Construction

Classical MAPF suites do not capture permanent geometric occupancy or multi-terminal pipe groups, and we are not aware of a public 3D multi-group pipe-routing suite with controlled difficulty and a constructive feasibility certificate. We therefore build instances synthetically.

Each instance is a triple $(\mathcal{E}, \{G_i\}_{i=1}^m, \mathcal{W})$: environment $\mathcal{E}$, m groups with face terminals and uniform radius r, and a conflict-free **witness** $\mathcal{W}$ proving generator-level feasibility. Solvers never see $\mathcal{W}$ as a hint; it is used for validation and as a *reference length*, not an optimality bound. Growing $\mathcal{W}$ first, then painting obstacles in its complement, yields known-feasible clutter without coupling the suite to any solver under test.

Pipeline. (1) Grow tree-like witness corridors with AxisRay or PlaneSlice (groups attach in round-robin). (2) Paint static obstacles outside thickened witnesses with PillarField or CorridorRack, never invading clearance r on $\mathcal{W}$. (3) Emit YAML (grid, obstacles, terminals, witness). The product of two corridor methods and two obstacle families gives **four datasets**, each with levels `easy`/`middle`/`hard` (Table 2; N=100 successes per cell, domain 32^3, r=0.5).

Table 2: Difficulty schedule (m groups $\times$ n_t terminals per group). Shared: domain 32^3, r=0.5.

Level	AxisRay ($m \times n_t$)	PlaneSlice ($m \times n_t$)
easy	3×3	3×3
middle	6×6	8×8
hard	9×9	12×12

Corridor generators. Both produce multi-terminal trees, not a single s–t path; inter-group corridors are obstacles for later growth. AxisRay (preferred) grows an axis-aligned polyline tree by LOS scans (no subdomain grid, no A*), with path/branch gaps and forced axis changes at turns. PlaneSlice grows on axis-aligned planes with *planar* A* only (no 3D fallback), using an $N \times N \times N$ subdomain layout and a plane-gap within each group. Pseudocode and clearance rules appear in Appendix A.

Obstacle generators. Witness cells are thickened into a keep-out set C; obstacles never invade C. PillarField places large AABBs until target density $\rho^\star = V_{\text{obs}}/V_0$ (we use $\rho^\star$=0.4), sampling seeds by distance to blocked cells. CorridorRack dilates witnesses by k Chebyshev steps (k=2 in reported runs) and meshes the complement as AABBs (density derived). Details in Appendix A.

5 Evaluation Protocol

Given $(\mathcal{E}, \{G_i\})$ without $\mathcal{W}$, a solver must return conflict-free paths for all topology tasks or fail within budget. Reported runs fix topology to pure MST and low-level search to A* (26-connected, Manhattan), and compare CBS / PBS / PP as in Sec. 3.

Primary metric. Success rate under wall-clock budget T (easy 30 s, middle 60 s, hard 120 s):

$$\text{SuccessRate} = \frac{1}{N}\big|\{i : \text{instance } i \text{ solved within } T\}\big|.$$

We also report failure reasons (`timeout` / `no_solution`) and success-vs-time curves. Fixed-budget success matches batch design use better than optimality claims we do not make.

Secondary metrics. Runtime on successes; on successes, cost_ratio = cost(sol)/cost($\mathcal{W}$) as a descriptive quality signal (*not* an optimality gap—$\mathcal{W}$ is constructive). Cost comparisons across algorithms use the common success set to limit selection bias.

Reproducibility. The suite, batch runners, and JSONL summaries ship with the `pipe_design` codebase (`gen_robust.sh`, `run_bench.sh`, `analyze_runs.py`). We intend to freeze these scripts with any public release of the benchmark.

6 Experiments

We evaluate CBS, PBS, and PP on the four datasets of Sec. 4 (N=100 instances per cell). Shared solver settings match Sec. 5: pure MST topology, A* / 26-connected / Manhattan, wall-clock budgets 30 s / 60 s / 120 s for easy / middle / hard. Tables report success rate under the level budget; hard Success@T curves use wall time with failures treated as $+\infty$.

Table 3: Success rate (%). Each cell: N=100. Budgets: easy 30 s, middle 60 s, hard 120 s.

Dataset	Level	CBS	PBS	PP
AxisRay×PillarField	easy	100	100	100
	middle	100	99	99
	hard	99	81	71
AxisRay×CorridorRack	easy	100	100	100
	middle	100	98	98
	hard	100	77	64
PlaneSlice×PillarField	easy	100	100	100
	middle	98	90	75
	hard	56	7	0
PlaneSlice×CorridorRack	easy	100	100	100
	middle	100	96	80
	hard	66	11	0

6.1 Success rate

Table 3 summarises success rates. On AxisRay, all three algorithms solve easy and middle almost completely; on hard, CBS remains near-perfect (99–100%) while PBS and PP drop to roughly 77–81% and 64–71%, respectively. On PlaneSlice, easy remains saturated, but middle already separates methods, and hard is strongly discriminating: CBS reaches 56% (PillarField) and 66% (CorridorRack), PBS only 7% / 11%, and PP solves *none* of the 200 hard instances. Obstacle family does not change the high-level ranking on PlaneSlice; both painters support the same ordering CBS $\gg$ PBS $\gg$ PP.

6.2 Runtime and Success@T

When successful, easy instances finish in tens of milliseconds (PP often ∼1 ms). On AxisRay hard, CBS/PBS median success times stay below 2 s; PP successes are much slower (median ∼38 s). On PlaneSlice hard, CBS median success time is on the order of 46–61 s, and many failures are timeouts—the budget is binding.

Table 4 reports hard Success@T at 5/30/60/100 s (budget in the runs is 120 s; the 100 s column is slightly conservative). AxisRay hard is largely decided by 5–30 s for CBS/PBS. PlaneSlice hard continues to accumulate CBS successes through 100 s (50–62%), while PBS plateaus near 7–9% and PP remains at 0%. PBS/PP gains from 60 s to 100 s are small: structure and dead orders dominate, not a missing few seconds.

6.3 Failures and path cost

On PlaneSlice hard, CBS failures are almost all timeouts (44 and 34 of 100). PBS mixes timeouts with a few `no_solution` outcomes; PP fails with a large share of both timeout and exhausted order neighbourhoods (∼33–34 `no_solution` per obstacle family). On AxisRay hard, PBS/PP failures are mostly `no_solution` rather than timeout, consistent with brittle orders once corridors interact.

Median cost_ratio on successes is typically below 1 (often ∼0.53–0.76 on hard non-trivial cells): constructive witnesses are feasible but loose. We treat cost_ratio as a descriptive length signal only, not as an optimality gap.

Table 4: Hard Success@T (%). Failures counted as $+\infty$ time.

Dataset	Algo	@5 s	@30 s	@60 s	@100 s
AxisRay×PillarField	CBS	96	99	99	99
	PBS	78	81	81	81
	PP	16	31	52	67
AxisRay×CorridorRack	CBS	94	100	100	100
	PBS	76	76	77	77
	PP	11	27	49	61
PlaneSlice×PillarField	CBS	0	8	28	50
	PBS	0	2	6	7
	PP	0	0	0	0
PlaneSlice×CorridorRack	CBS	0	17	43	62
	PBS	0	1	5	9
	PP	0	0	0	0

7 Related Work

Multi-agent pathfinding. Grid MAPF uses high-level controllers such as CBS [1], PBS [2], and priority planning [3]. Agents occupy cells temporarily; conflicts are vertex- or edge-based at explicit timesteps. Stern et al. [4] unify MAPF terminology and publish grid benchmarks for spatiotemporal planning. We reuse the same *control patterns*—constraint trees, priority DAGs, total orders—but under permanent geometric occupancy, dual-witness clearance conflicts, and multi-terminal groups. Our suite targets design-time 3D layout rather than concurrent motion on MovingAI-style MAPF maps [4].

3D pipe routing and MAPF transfer. Belov et al. [5] draw the analogy between MAPF blocked cells in x–y–t and pipe routing in x–y–z, and adapt CBS and PBS to industrial PR instances with one start and one goal per pipe. They evaluate on plant-scale and smaller synthetic cases, but do not provide a public multi-group benchmark with constructive feasibility witnesses or a systematic comparison including priority planning. We extend the MAPF-to-routing line to *grouped* multi-terminal topology, explicit permanent occupancy, and a reproducible four-dataset suite (Sec. 4).

Automatic pipe routing. Blokland et al. [6] survey automatic pipe routing for large architectures (ships, chemical plants), where sequential CAD-style routing and mixed-integer or sampling methods dominate and full automation remains difficult. That literature rarely expresses cross-pipe interaction as MAPF-style branching and seldom releases controlled 3D instance generators with certificates. Our contribution is closer to a MAPF benchmark paper [4] than to a single-application optimiser: fixed protocol, open scripts, and baseline success rates on N=100 instances per cell.

Application context. Industrial PR motivates the abstraction [5, 6]; our intro uses AM-oriented multi-group valve layout as a representative case. Reported experiments remain on constructive 32^3 grids so that difficulty and metrics are comparable across CBS, PBS, and PP.

8 Discussion

The benchmark is not uniformly hard. AxisRay easy and middle saturate for all three controllers; only on hard do PBS and PP fall behind CBS, and even then CBS stays near 100%. PlaneSlice—especially hard with many groups—is where methods separate: CBS reaches 56–66% success while PBS drops to single digits and PP solves none of the 200 hard instances under the same budget. That pattern matches the problem structure: once prior pipes are permanent obstacles, order- and priority-based search can lock in early commitments, whereas CBS can branch on geometric conflicts and add localized negative constraints. Obstacle family (PillarField vs. CorridorRack) does not change the ranking on PlaneSlice, which suggests the stress test comes from corridor layout and group count rather than a particular clutter model.

Several scope limits follow directly. Instances are synthetic 32^3 grids with uniform radius; we do not evaluate on B-rep valve bodies, multi-radius mixtures, or manufacturing criteria such as bend limits or supportability. Constructive witnesses certify generator feasibility only—cost_ratio below 1 must not be read as an optimality gap. We report CBS, PBS, and PP as baselines under one fixed protocol, not as an exhaustive comparison to commercial CAD routers or continuous shape optimisers; post-layout smoothing remains orthogonal follow-on work.

9 Conclusion

We formalized multi-group 3D pipe routing under permanent geometric occupancy, introduced a constructive benchmark with four datasets and controlled difficulty, and measured fixed-budget success for CBS, PBS, and priority planning on N=100 instances per cell. The suite and reported runs are intended as a reproducible starting point for stronger geometric multi-group planners and for coupling discrete layout search to manufacturing-aware refinement.

References

[1] G. Sharon, R. Stern, A. Felner, and N. R. Sturtevant. Conflict-based search for optimal multi-agent pathfinding. In *Proc. AAAI*, pages 563–568, 2012.

[2] H. Ma, D. Harabor, P. J. Stuckey, J. Li, and S. Koenig. Searching with consistent prioritization for multi-agent path finding. In *Proc. AAAI*, pages 7643–7650, 2019.

[3] A. Felner, R. Stern, S. E. Shimony, E. Boyarski, M. Goldenberg, G. Sharon, N. Sturtevant, and S. Koenig. Search-based optimal solvers for the multi-agent pathfinding problem: summary and challenges. In *Proc. SoCS*, pages 29–36, 2017.

[4] R. Stern, N. R. Sturtevant, A. Felner, S. Koenig, H. Ma, T. Walker, J. Li, D. Atzmon, L. Cohen, T. K. S. Kumar, E. Boyarski, and R. Barták. Multi-agent pathfinding: Definitions, variants, and benchmarks. In *Proc. SoCS*, volume 10, pages 151–158, 2019.

[5] G. Belov, W. Du, M. Garcia de la Banda, D. Harabor, S. Koenig, and X. Wei. From multi-agent pathfinding to 3D pipe routing. In *Proc. AAAI*, pages 10257–10265, 2020.

[6] M. Blokland, R. D. van der Mei, J. F. J. Pruyn, and J. Berkhout. Literature survey on automatic pipe routing. *Operations Research Forum*, 4(35), 2023.

Algorithm 1 AxisRay: generate witness corridors

Require: domain size, radius r, groups $\{n_t^{(g)}\}$, waypoint budgets, gaps
Ensure: per-group corridors or **fail**

Occ $\leftarrow \emptyset$; WP$[g] \leftarrow \emptyset$ for all groups g
while some group needs more face terminals **do**
  for group g in round-robin order **do**
    if g has no corridor **then**
      sample face terminal s; grow seed chain with `init_waypoints_num`-2 interior LOS segments
      final LOS segment to a domain face; record waypoints
      on fail: rollback Occ / paths for this attempt
    else
      pick branch point p on g's tree with dist$(p, \text{WP}[g]) >$ `branch_gap`
      pick side-exit direction (not along the current edge axis)
      grow attach chain (`attach_waypoints_num`) to a new face
      on fail: rollback this attach
    end if
  end for
  if no progress after max attempts **then**
    return fail
  end if
end while
return corridors

A Generator Details

This appendix records constructive generators used in Sec. 4. It is intended for reproducibility; the main text only needs the high-level behaviour.

A.1 AxisRay

AxisRay uses LOS scans with rollback on failed seed/attach attempts. Hard rules include: no entry into written path cells; Chebyshev gap `path_gap` except along the spine of the edge being left (requires `branch_gap`>`path_gap`); consecutive segments change axis; branch points stay farther than `branch_gap` from existing waypoints.

A.2 PlaneSlice

Every search is restricted to the active axis plane (2D); there is no fallback to full 3D A*. Same-group corridors off the plane are ignored during search; other groups on the plane are obstacles. Same-axis planes within a group obey a plane-gap.

A.3 PillarField and CorridorRack

Let C be witness cells thickened by `corridor_margin` (and r). Eligible volume V_0 is the complement of C at paint start.

Algorithm 2 PlaneSlice: generate witness corridors

Require: domain, r, subdomain N, plane gap, targets $\{n_t^{(g)}\}$
Ensure: per-group corridors or **fail**

build subdomain partition; Planes$[g] \leftarrow \emptyset$
while some group needs more terminals **do**
  for group g in round-robin order **do**
    if g empty **then**
      sample valid axis-plane P; place two face terminals s, t on P
      planar A* on P only; record path and P
    else
      sample plane type $\neq$ last plane; derive P from a cell of g
      multi-source starts $\leftarrow$ all cells of g lying on P
      pick new face goal on P; planar multi-source A* on P
      on success: append path, record P
    end if
  end for
end while
return corridors

Algorithm 3 PillarField: paint AABB obstacles

Require: instance with witness, $\rho^\star$, edge bounds, `dist_cap`

mark thickened witness as blocked; build Chebyshev distance field $d(\cdot)$
$V_0 \leftarrow$ count of free cells; $V_{\text{obs}} \leftarrow 0$
while $V_{\text{obs}}/V_0 < \rho^\star$ and progress **do**
  sample seed c with weight $\propto \min(d(c), \texttt{dist_cap})$
  expand AABB from c (greedy / isometric); reject if too thin
  among candidates, commit the largest-volume box into blocked
  update $d(\cdot)$ from newly blocked cells; V_{obs} += volume
end while
emit committed boxes as static obstacles

Algorithm 4 CorridorRack: complement of dilated witnesses

Require: instance with witness, dilate steps k

$F \leftarrow$ Chebyshev dilate of all witness cells by k
ObsCells $\leftarrow$ domain $\setminus F$
merge ObsCells into large AABBs; append to static obstacles
record derived $\rho \leftarrow V_{\text{obs}}/V_{\text{domain}}$